P6 evaluation: concrete deliverables and implementation

Updated: 5 November 2026. Branch: `P6/evaluation`. Shared schemas/split mappings remain `v0-proposed`.

This package converts saved system outputs into reproducible evaluation tables, and P3's submitted labels into audited gold labels. P6 will not run the end-to-end evaluation, and who will is unknown. System training, human adjudication and team freeze decisions are separate responsibilities.

Deliverables and how the team uses them

P6 deliverable	Input	Output	Used for
Annotation audit and gold export	Pinned P3 sheets/pool; disclosed review of the 188 disagreements or third-person adjudication; real human review; exact corpus	`agreement.json`, `adjudication.csv`, `final_labels.jsonl`, label manifest, preserved original judgments	Q4.3 agreement; gold input to retrieval scoring
Retrieval scoring	P3 saved rankings/questions/pairs; finalized gold; exact corpus	`scores.jsonl`, `retrieval.csv`, summary and provenance	Q3 four-method comparison; no historical model rerun
Evidence-mapping recall	Original static/adaptive retrieval records; P3 mapping/manifest; matching corpus	`evidence_recall.csv`, per-question recall and denominators	Static@5/@8/@10 vs adaptive, separate from pool scores
QA scoring	Selected questions; saved P4/P5 answers; QASPER references	Per-question EM/F1, answerable/unanswerable results, failures and coverage; `qa.csv`	Q4.4 answer quality
RAGAS scoring	Questions/answers; exact rendered contexts; references; configured judge	Faithfulness, response relevance, context precision, explicit statuses and valid counts; `ragas.csv`	Q4.5 evidence-grounded answer quality
Performance summary	Saved request timings; measured batch wall time; optional usage/prices	`performance.csv`, optional `usage_cost.csv`	Q4.6 latency/throughput/cost; Q2.2 query speed

P6 deliverable	Input	Output	Used for
Matched comparison	Two per-question score files, one arm each	`ablation.csv` with right-minus-left differences and paper-clustered intervals	Q5 controlled comparisons
Instructions and notebook	Commands, contracts and worked synthetic examples	Team handoff and guided notebook	Teammates reproduce all steps

What is implemented

One entry point: `python -m scripts.run_p6_evaluation --help`.

Command	Implementation
`export-p3`	Export the pinned frozen pool/rankings from the local P3 Git ref, after validating sheets/manifests
`audit-labels`	Check pool hashes, original sheet bodies, identities and completed labels; report independent-label agreement and prepare adjudication CSV
`finalize-labels`	Validate reviewed human provenance and exact corpus; apply a disclosed review (`--disagreements resolved`) or validate third-person decisions (`--disagreements adjudicate`); export gold and a scorer-compatible manifest
`retrieval`	Validate frozen P3 ranking bytes/IDs/ranks, label scope and corpus; reuse existing metric arithmetic at $K = 1, 3, 5$
`evidence`	Score static ranked cutoffs and adaptive cumulative rounds against verified P3 mapping; report failure-inclusive and P4-compatible successful-only means
`qa`	Read original QASPER or normalized references, adapt P4/P5 or normalized predictions, score and export
`contexts`	Reconstruct P5 evidence with an explicitly reviewed prompt configuration and matching saved prompt version
`ragas`	Validate inputs without a model by default; optional explicit live RAGAS 0.2.15 backend, cache and raw local judge trace
`performance`	Aggregate internal or client timings, measured batch throughput, optional token ledger/pricing
`compare`	Match question/paper IDs and bootstrap paired metric differences by paper

Core reusable files are under `src/evaluation/`. Existing producer interfaces are unchanged. Every successful command creates a new output directory and records input/output/source hashes. Existing outputs are never overwritten; failed commands do not publish a partial output bundle. Live judge caches may survive failed jobs intentionally.

Run order and completion criteria

1. **Audit P3 submissions.** Review changes and true human identities/independence. Latest audited commit `1a1dc15b824590a39c80ea11a568fbe5814a926e`: 1,853 pairs, 3,706 labels, 1,665 matches (89.854%), 188 disagreements. Method names were visible; do not claim system-blind labeling. Earlier notebook snapshots are historical.
2. **Resolve disagreements.** Selected policy: a disclosed review (`--disagreements resolved`) gives all 188 disagreements a final label, so gold judges all 1,853 pairs. This is not third-person adjudication, and reports must carry the disclosure in the handoff. The adjudication alternative remains supported: a third person outside each original pair supplies final label/reason. P3/P6 confirm use history and role; export validated gold. Initial agreement is never recomputed from adjudicated labels.
3. **Score saved retrieval.** Export pinned rankings, check exact corpus and score $K = 1, 3, 5$. Report pool-relative recall/nDCG and judged coverage, not exhaustive paper recall.
4. **Run a development QA pilot.** Owners use their existing runners; the evaluation lead (unknown) scores outputs and verifies answer/reference/context adapters. No reference or gold evidence enters generation.
5. **Configure and smoke-test the judge.** Supply approved endpoint/models/budget, install optional packages separately, validate inputs, then explicitly run live RAGAS on development examples.
6. **Freeze and run final configurations.** Team confirms eligible questions, actual use history, prompt/model/index/configuration hashes and named batch operator. Save outputs before offline reference joins. Existing agent runner refuses official test inputs; do not bypass the guard with `--items`.
7. **Score and hand over tables.** Include metric denominators, missingness, refusals/errors, hardware/caching/concurrency and reviewed interpretation. Each owner reviews claims about their module.

What still requires people or real runtime inputs

Needed	Provider	Blocks
Submission independence/history review (188 disagreements resolved by disclosed review, pending team sign-off)	Actual annotators/adjudicators, P3/P6	Final gold and final retrieval table
Full corpus matching the pool	P3	Verified gold/corpus identity
Final set and freeze/use review, named inference operator	Unknown	Final evaluation claims and final inference
Saved real predictions, pinned rendering configuration or exact context export	P4/P5	Real QA/RAGAS results
Approved judge/embedding access and budget	Unknown	Live RAGAS verification and results
Measured batch interval/client times/usage	Runtime operator	True throughput, HTTP latency and

Needed	Provider	Blocks
		cost

The code is implemented; these items are not automatically completed by installing it. The handout does not assign P1–P6 roles: this is the proposed concrete P6 package for team review.

Important interpretation rules

- P3 relevance labels identify useful passages. QA reference answers are different input data.
- `elapsed_s` excludes generation; `answer_s` adds generation. Their sum is internal pipeline time, not HTTP/UI latency.
- Critic context and all retrieved passages are not automatically answer-generator context. Reconstruction requires the exact original P5 configuration/version and owner review.
- Static/adaptive tests a combined adaptive module. Separate rewriting/reranking claims require the Q5 2×2 ablation. Use larger static-depth controls to assess gains from reading more context.
- The existing R1–R4/G0–G5 ladder in `docs/EVALUATION.md` remains the shared plan. All final held-out decisions/schemas remain subject to affected-owner confirmation.

Start with the team usage handoff. Learn the calculations in the walkthrough notebook and metric reference. The older annotation notebook is a reference; use the new CLI for the completed manifest contract.